Introduction to Docker

Application Development Scenario: Before Containers

- Developer-1 uses MAC for coding
- Developer-2 uses Linux for coding
- Both require the softwares like PostgresSQL and Redis to be installed on their machines
- Installation process is different
  - Chances of error during installation
- This process can cause more issues as the number of machines increase

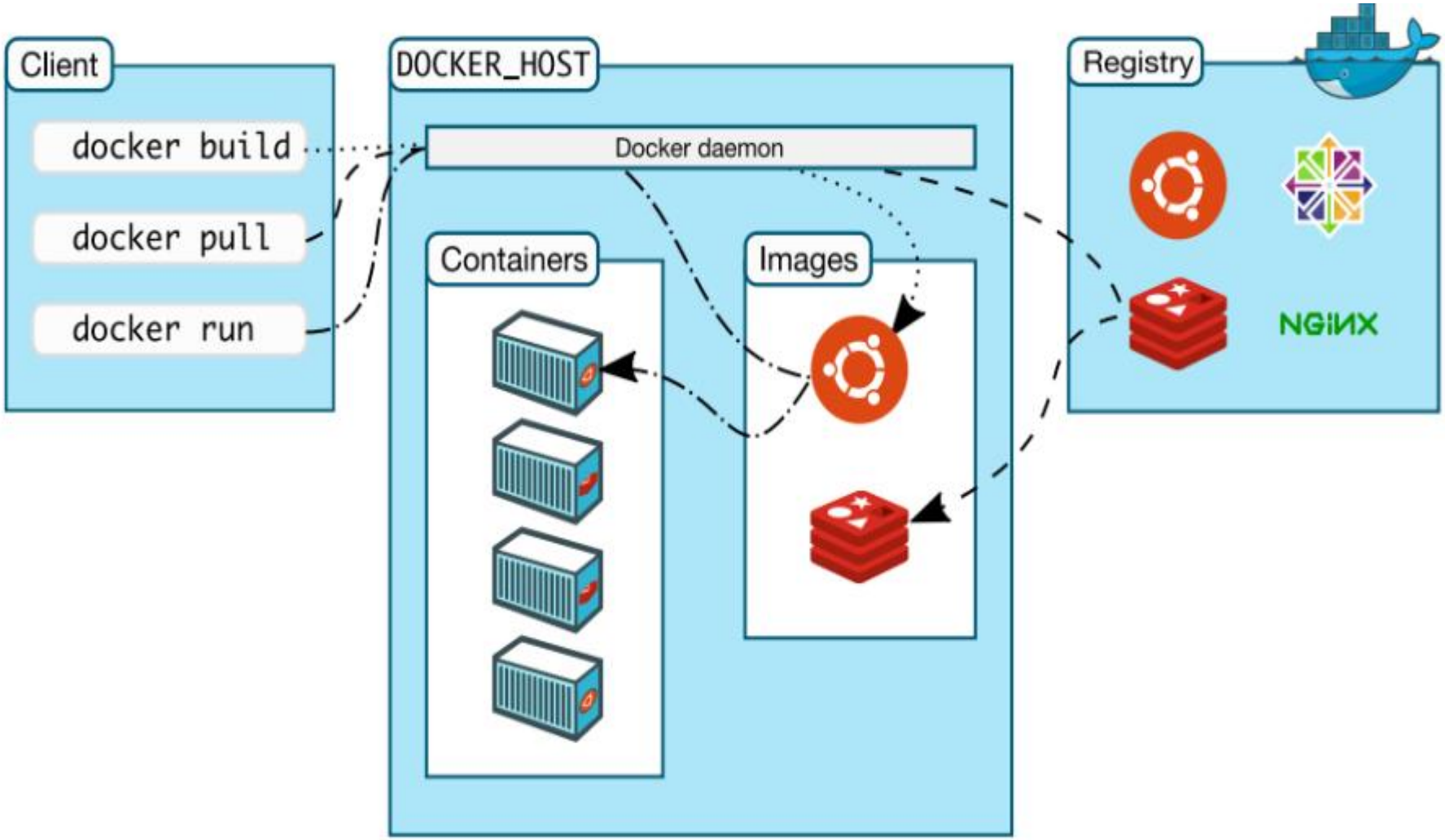
Application Development Scenario: After Containers

- Container includes everything needed to be installed as one isolated environment
- Developers can just download the container and run it on their individual machines
- Simple docker commands can be used to download the container
- This makes setting up local environment much easier and efficient
- Different versions of same application can be executed

Docker

- Docker is a Linux-based, open-source container management service
- “Develop once and run anywhere”
- Develop applications, ship them into container and deploy it anywhere
- Developed by Solomon Hykes
- First released in March 2013
- Written in Go language
- Containerd is the official runtime of Docker

Docker Architecture



Components of Docker Architecture

- Docker Client
- Docker Daemon
- Docker Images
- Docker Containers
- Docker Registry (Docker Hub)

Docker Image

- A Docker image is a
  - Lightweight
  - Standalone
  - Executable package of software

- It includes everything needed to run an application
  - Code, runtime, system tools, system libraries and settings

Docker Image

- An image is a read-only template with instructions for creating a Docker container
- Often, an image is based on another image, with some additional customization
- For example, you may build an image which is based on the ubuntu image, but install the Node.js server and your application, as well as the configuration details needed to make your application run

Docker Image

- You can create your own images
- You can also use those images, created by others and published in a registry
- To build your own image, you create a Dockerfile with a simple syntax for defining the steps needed to create a image and run it.

Docker Image

- Each instruction in a Dockerfile creates a layer in the image
- When you change the Dockerfile and rebuild the image, only those layers which have changed are rebuilt
- This is part of what make images so lightweight, small, and fast, when composed to other virtualization technologies

Container

- Container is running instance of image
- Works as an Isolated process
- You can create, start, stop, move, or delete a container using the Docker API or CLI
- You can
  - Connect a container to one or more networks
  - Attach storage to it, or
  - Create a new image based on its current state

Container

- A container is a standard unit of software that packages up
  - Code and
  - All its dependencies

- So the application runs quickly and reliably among different computing environments

Container

- Container is relatively well isolated from other containers and its host machine
- You can control how isolated a container's network, storage, or other underlying subsystems are from other containers or from the host machine
- A container is defined by its image as well as any configuration options you provide to it when you create or start it
- When a container is removed, any changes to its state that are not stored in persistent storage disappear

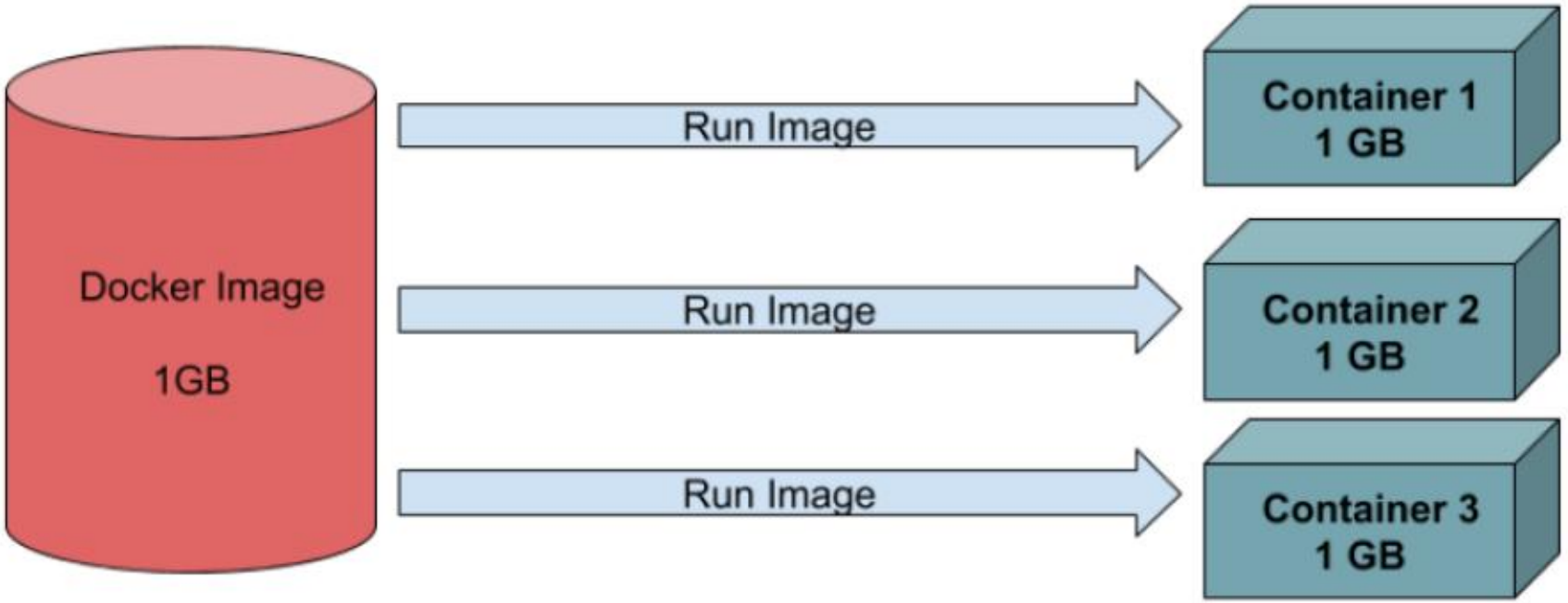
Docker Image vs. Container

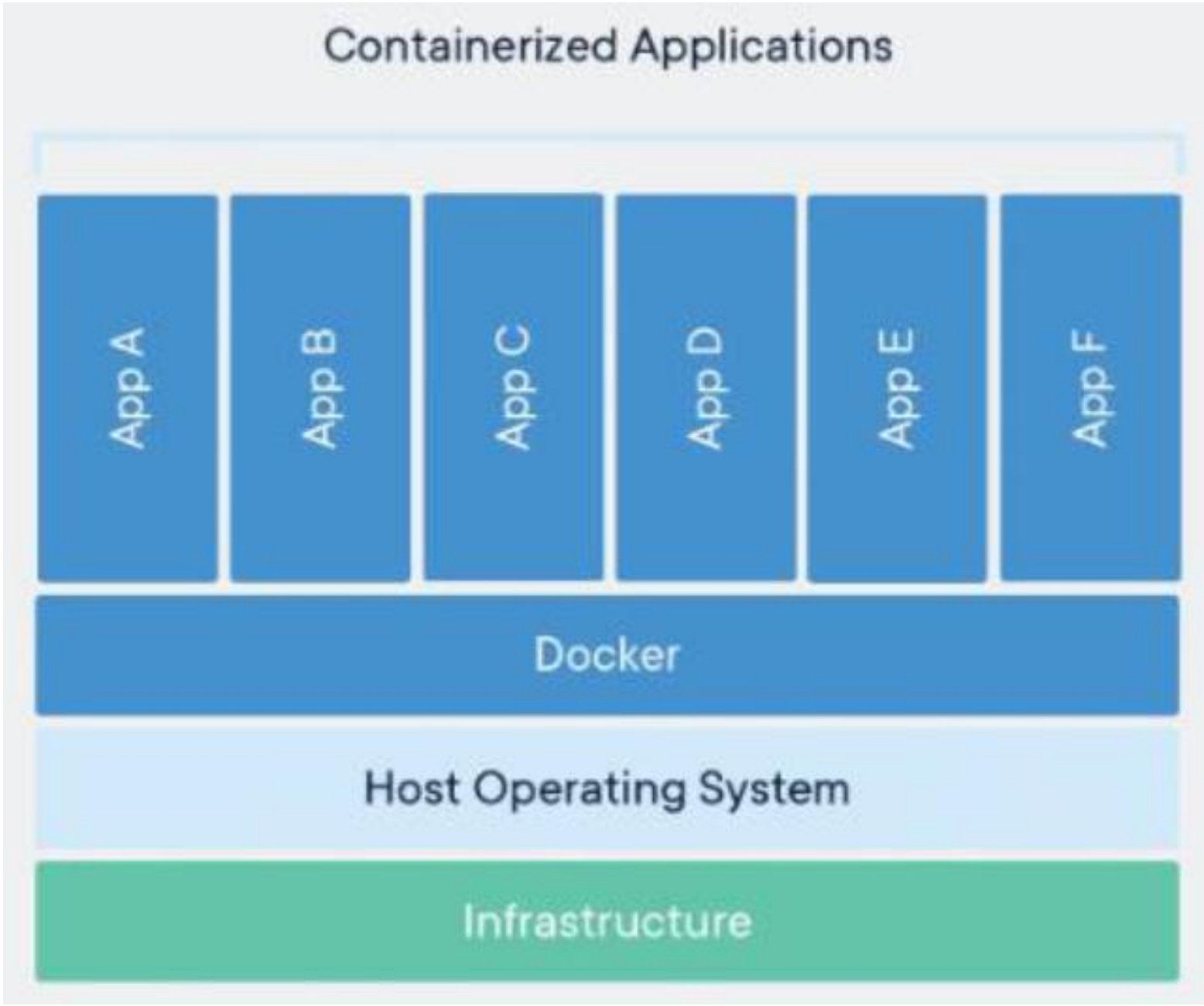
- Docker images become containers when they run on Docker engine
- Available for both Linux and Windows (Docker Desktop), containerized software will always run the same, regardless of the infrastructure
- Containers isolate software from its environment and ensure that it works uniformly despite differences for instance between development and staging

Docker Image vs. Container

Feature	Docker Image	Docker Container
Nature	Template	Running instance
State	Static	Dynamic
Modifiable	No (read-only)	Yes (while running)
Stored In	Registry	Host Machine
Example	nginx image	Running nginx container

Docker Image vs. Container




Containerized Applications
App A
App B
App C
App D
App E
App F
Docker
Host Operating System
Infrastructure

Container Characteristics

- Multiple containers can run on the same machine
- They share the OS kernel with other containers
- Each runs as isolated processes in user space
- Containers take up less space (in MBs) than VMs (in GBs)
- It can handle more applications and require fewer VMs and Operating systems

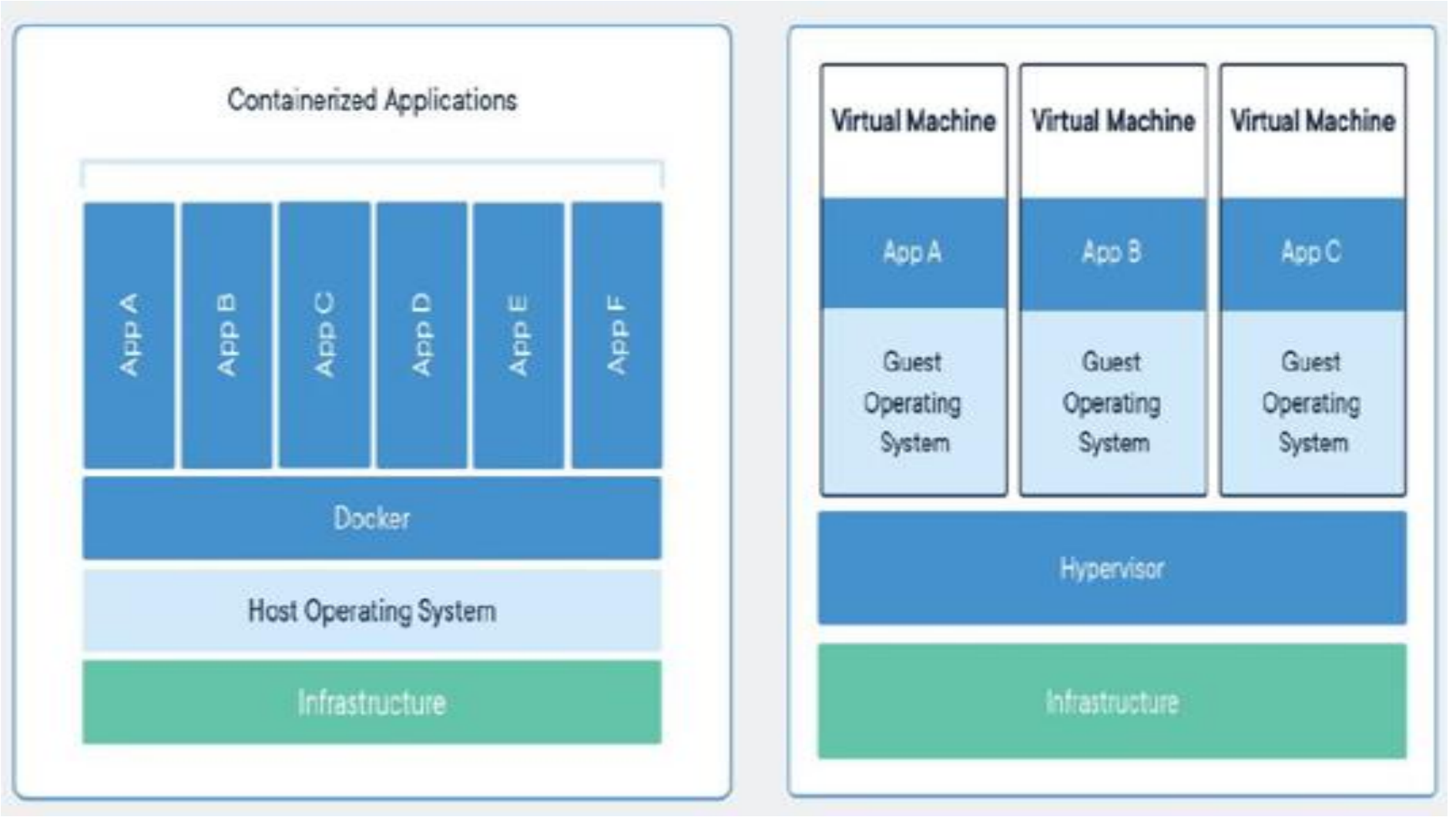
Virtual Machine

- A Virtual Machine (VM) is a compute resource that uses software instead of a physical computer to run programs and deploy apps
- One or more virtual “guest” machines run on a physical “host” machine
- Each virtual machine runs its own operating system from the other VMs, even when they are running on the same host

Virtual Machine Characteristics

- VMs are an abstraction of physical hardware turning one into many servers
- The hypervisor allows multiple VMs to run on a single machine
- Each VM includes a full copy of an OS, the application, necessary binaries and libraries (taking more space, in GBs)
- VMs can also be a slow boot

Container vs. Virtual Machine



The Docker Daemon

- The Docker daemon (dockerd) listens for Docker API requests and manages Docker objects such as
  - Images, containers, networks and volumes

- A daemon also communicate with other daemons to manage Docker services

The Docker Client

- The Docker client (docker) is the primary way that many Docker users interact with Docker
- Using commands such as docker run, the client sends these commands to dockerd, which carries them out
- The docker command uses the Docker API
- The Docker client can communicate with more than one daemon

The Docker Desktop

- Docker Desktop is an easy-to-install application for Mac, Windows or Linux environment
- It enables you to build and share containerized applications and microservices
- Docker Desktop includes the Docker daemon (dockerd), the Docket client (docker), Docker Compose, Docker Client Trust, Kubernetes and Credential Helper

The Docker Registry

- It stores Docker images
- Docker Hub is a public registry that anyone can use
- Docker is configured to look for images on Docker Hub by default
- You can even create your own private registry
- When you use the docker pull or docker run commands, the required images are pulled from your configured registry
- When you use docker push command, your image is pushed to your configured registry

Install Docker Desktop

Docker Desktop for Mac

Download (Apple Silicon) | Download (Intel) | Install instructions

Docker Desktop for Windows

Download | Install instructions

Docker Desktop for Linux

Install instructions

```
C:\Users\CEDDU>docker version
Client:
 Version:           29.2.1
 API version:       1.53
 Go version:        go1.25.6
 Git commit:        a5c7197
 Built:             Mon Feb  2 17:20:16 2026
 OS/Arch:           windows/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.61.0 (219004)
 Engine:
  Version:          29.2.1
  API version:      1.53 (minimum version 1.44)
  Go version:       go1.25.6
  Git commit:       6bc6209
  Built:            Mon Feb  2 17:17:24 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.2.1
  GitCommit:        dea7da592f5d1d2b7755e3a161be07f43fad8f75
 runc:
  Version:          1.3.4
  GitCommit:        v1.3.4-0-gd6d73eb8
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

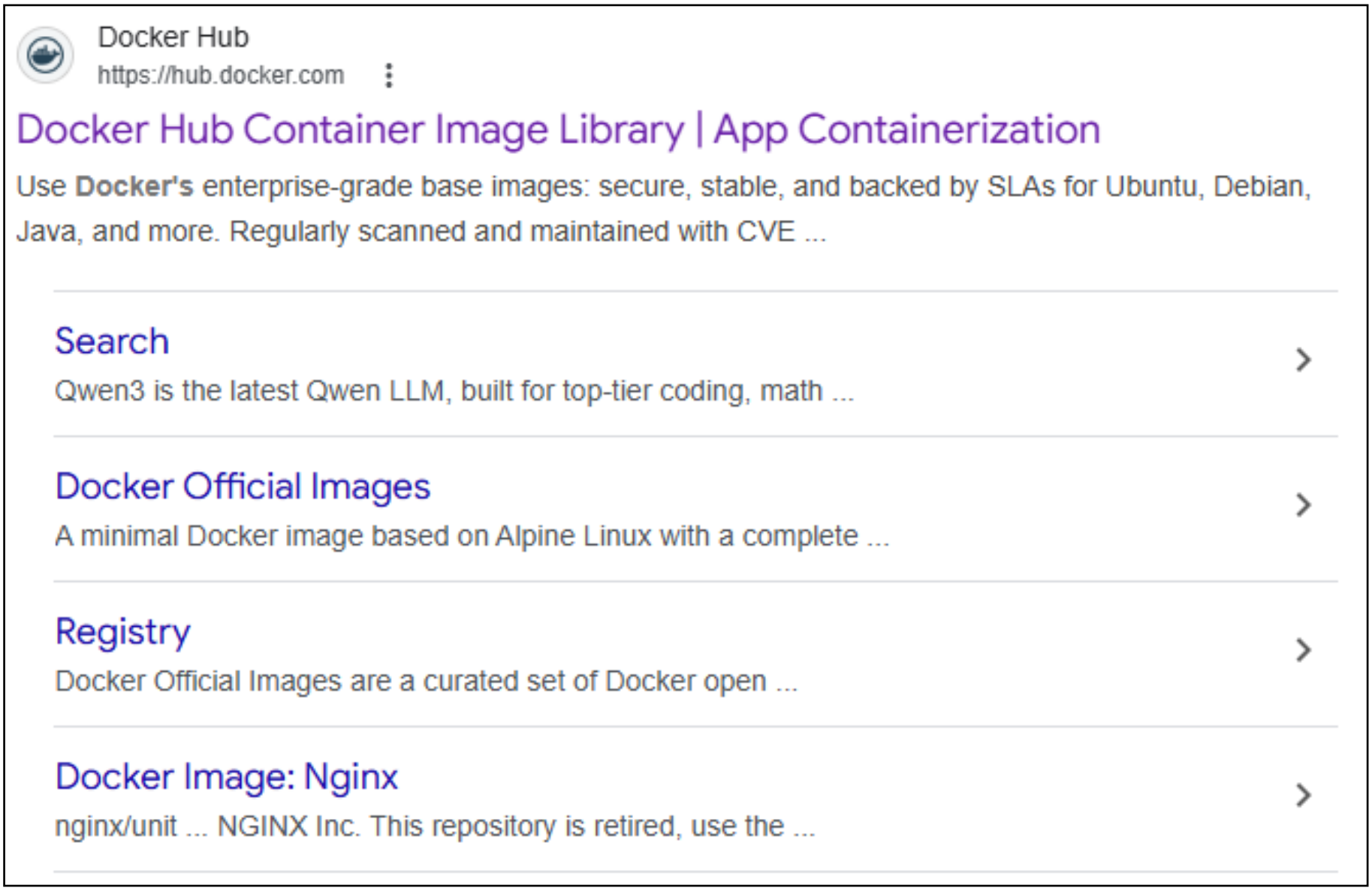

Docker Hub
https://hub.docker.com
Docker Hub Container Image Library | App Containerization
Use Docker's enterprise-grade base images: secure, stable, and backed by SLAs for Ubuntu, Debian, Java, and more. Regularly scanned and maintained with CVE ...
Search
Qwen3 is the latest Qwen LLM, built for top-tier coding, math ...
Docker Official Images
A minimal Docker image based on Alpine Linux with a complete ...
Registry
Docker Official Images are a curated set of Docker open ...
Docker Image: Nginx
nginx/unit ... NGINX Inc. This repository is retired, use the ...

IMAGE + 1 MORE

memcached Docker Official Images

Free & open source, high-performance, distributed memo...

Pulls	1B+
Stars	2429
Last Updated	18 days

IMAGE + 1 MORE

nginx Docker Official Images

Official build of Nginx.

Pulls	1B+
Stars	21186
Last Updated	13 days

IMAGE + 1 MORE

busybox Docker Official Images

Busybox base image.

Pulls	1B+
Stars	3487
Last Updated	18 days

IMAGE

alpine Docker Official Images

A minimal Docker image based on Alpine Linux with a complete...

Pulls	1B+
Stars	11469
Last Updated	24 days

Databases & storage

IMAGE + 1 MORE

postgres Docker Official Images

The PostgreSQL object-relational database system provides...

Pulls	1B+
Stars	14792
Last Updated	2 days

IMAGE + 1 MORE

mysql Docker Official Images

MySQL is a widely used, open-source relational database...

Pulls	1B+
Stars	16071
Last Updated	1 day

IMAGE

neo4j Docker Official Images

Neo4j is a highly scalable, robust native graph database.

Pulls	100M+
Stars	1361
Last Updated	2 days

IMAGE + 1 MORE

mongo Docker Official Images

MongoDB document databases provide high availability and eas...

Pulls	1B+
Stars	10699
Last Updated	about 1 hour

MySQL

DATABASES & STORAGE

Overview Tags

Quick reference

- **Maintained by:**
  the Docker Community and the MySQL Team
- **Where to get help:**
  the Docker Community Slack, Server Fault, Unix & Linux, or Stack Overflow

Supported tags and respective `Dockerfile` links

- `9.6.0`, `9.6`, `9`, `innovation`, `latest`, `9.6.0-oraclelinux9`, `9.6-oraclelinux9`, `9-oraclelinux9`, `innovation-oraclelinux9`, `oraclelinux9`, `9.6.0-oracle`, `9.6-oracle`, `9-oracle`, `innovation-oracle`, `oracle`
- `8.4.8`, `8.4`, `8`, `lts`, `8.4.8-oraclelinux9`, `8.4-oraclelinux9`, `8-oraclelinux9`, `lts-oraclelinux9`, `8.4.8-oracle`, `8.4-oracle`, `8-oracle`, `lts-oracle`

Tag summary

Recent tags

oraclelinux9

Content type

Image

Digest

sha256:e5dc14f6e...

Size

254 MB

Last updated

1 day ago

Docker: pull command

- This command allows to pull any image which is present in the official registry of docker, Docker hub.

- By default, it pulls the latest image, but you can also mention the version of the image.

docker pull <image_name>

```
C:\Users\CEDDU>docker pull mysql
Using default tag: latest
latest: Pulling from library/mysql
658630c937c6: Pull complete
357926362d31: Pull complete
61941d436e88: Pull complete
f3702ac323ca: Pull complete
4a629f1008ff: Pull complete
c89a19cdad3b: Pull complete
8314ff001dec: Pull complete
ab1b8ea72826: Pull complete
74a8e4bbd9fe: Pull complete
00af5f73db53: Pull complete
1dc8eadaef55: Download complete
b222ef59b124: Download complete
Digest: sha256:e5dc14f6e01c3e577e669337d2855c6d1561b30d8ef2c592e63e4e8a9a52650a
Status: Downloaded newer image for mysql:latest
docker.io/library/mysql:latest
```

Docker: run command

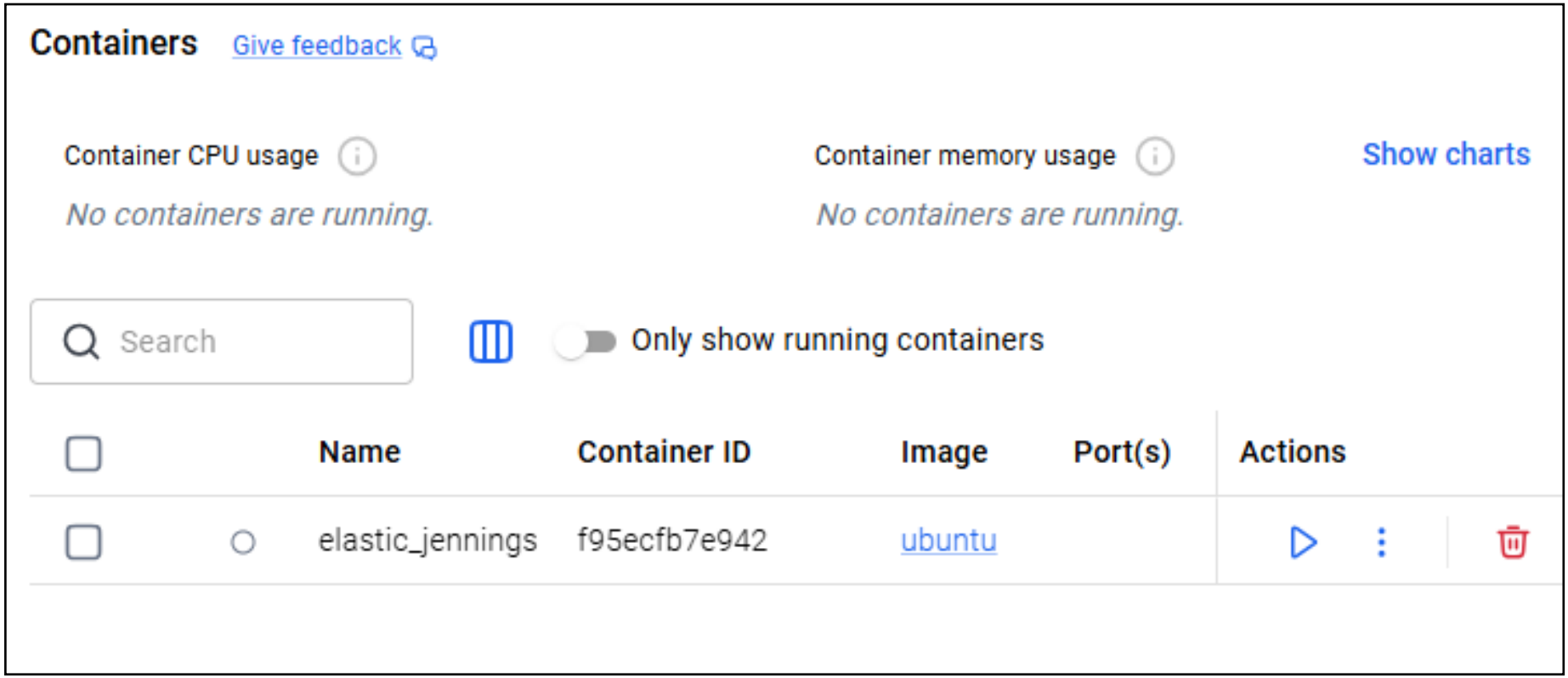
- This command is used to run a container from an image.
- The docker run command is a combination of the docker create and docker start commands.
- It creates a new container from the image specified and starts that container.
  - if the docker image is not present, then the docker run pulls that.

docker run <image_name>

- To give name of container

docker run --name <container_name> <image_name>

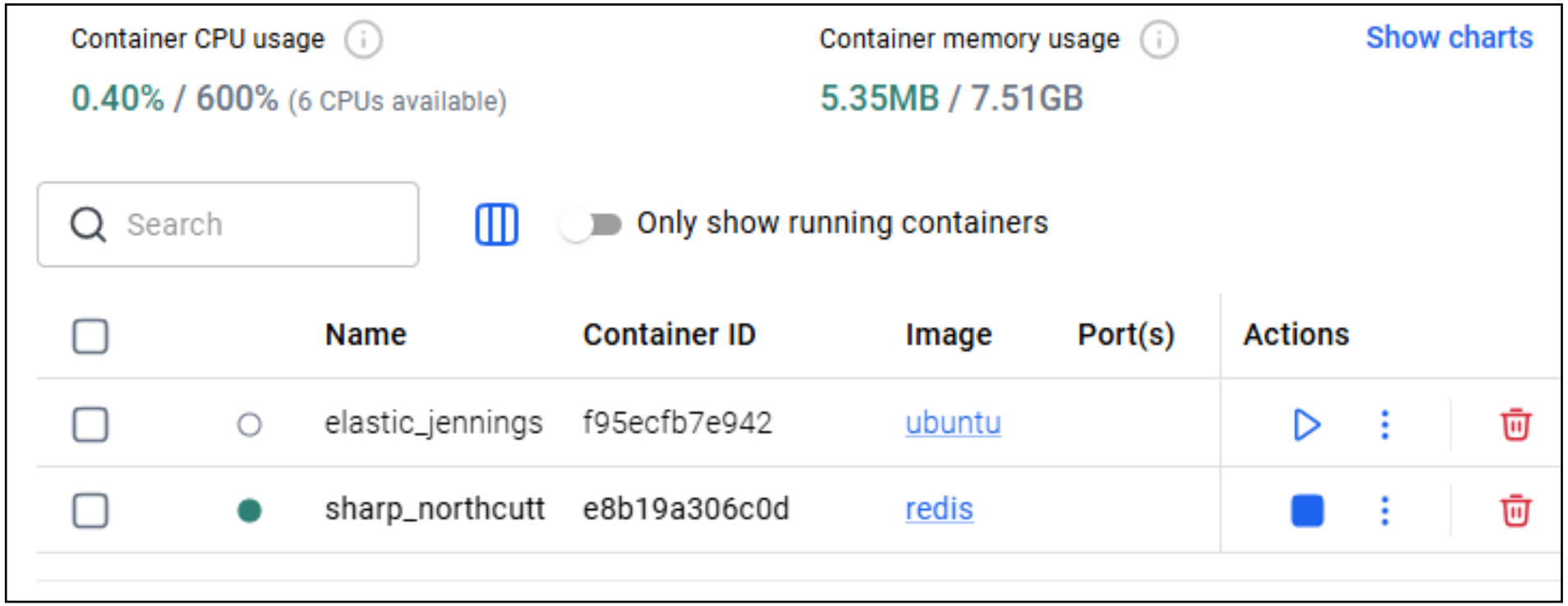
```
C:\Users\CEDDU>docker run ubuntu
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
01d7766a2e4a: Pull complete
fd8cda969ed2: Download complete
Digest: sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203eab2807c7e495eff4f9
Status: Downloaded newer image for ubuntu:latest
```


Containers
Give feedback
Container CPU usage
No containers are running.
Container memory usage
No containers are running.
Show charts
Search
Only show running containers
Name
Container ID
Image
Port(s)
Actions
elastic_jennings
f95ecfb7e942
ubuntu

Docker: run Command

```
C:\Users\CEDDU>docker run redis
Unable to find image 'redis:latest' locally
latest: Pulling from library/redis
4a43643650f6: Pull complete
35b667b631ff: Pull complete
6397ab563848: Pull complete
0c8d55a45c0d: Pull complete
4f4fb700ef54: Pull complete
a15985f29cdf: Pull complete
bbed7540f434: Pull complete
14bb7a3bd12b: Download complete
c9d74f9db85e: Download complete
Digest: sha256:7b6fb55d8b0adcd77269dc52b3cfffe5f59ca5d43dec3c90dbe18aacce7942e1
Status: Downloaded newer image for redis:latest
Starting Redis Server
```

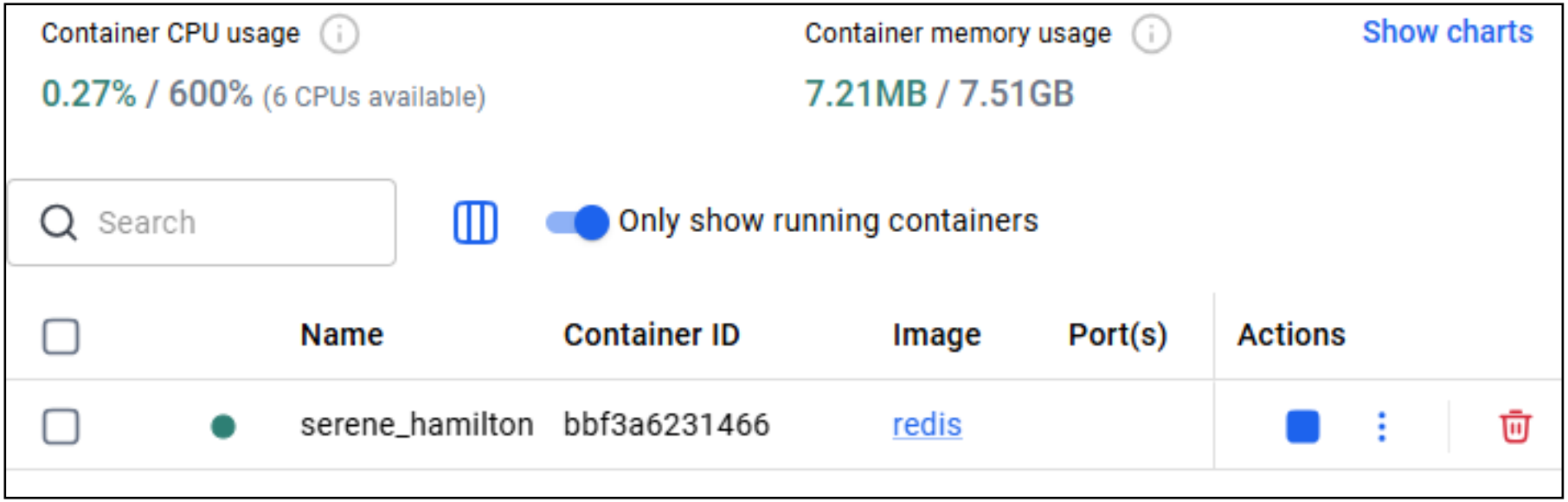
Docker: run Command



Docker in Detached Mode

```
C:\Users\CEDDU>docker run -d redis
bbf3a623146642d5103b41040eba1b0feb040074fd66a0f0e050f262f573fc17

C:\Users\CEDDU>
```



Docker: stop Command

```
C:\Users\CEDDU>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS      NAMES
bbf3a6231466   redis     "docker-entrypoint.s…"   9 minutes ago   Up 9 minutes   6379/tcp   serene_hamilton

C:\Users\CEDDU>docker stop bbf3a6231466
bbf3a6231466

C:\Users\CEDDU>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Users\CEDDU>
```

Docker: start Command

```
C:\Users\CEDDU>docker start bbf3a6231466
bbf3a6231466

C:\Users\CEDDU>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS      NAMES
bbf3a6231466   redis     "docker-entrypoint.s…"   11 minutes ago  Up 3 seconds   6379/tcp   serene_hamilton


C:\Users\CEDDU>
```

Docker: ps command

- This command (by default) shows a list of all the running containers.
- We can use various flags with it.

-a flag: shows all the containers, stopped or running.

-l flag: shows us the latest container.

-q flag: shows only the Id of the containers.

docker ps –a ---- To view all containers

```
C:\Users\CEDDU>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED             STATUS
   NAMES
bbf3a6231466   redis     "docker-entrypoint.s…"   13 minutes ago      Up 2 minutes
   serene_hamilton
e8b19a306c0d   redis     "docker-entrypoint.s…"   16 minutes ago      Exited (0) 13 minutes ago
   sharp_northcutt
f95ecfb7e942   ubuntu    "/bin/bash"              About an hour ago   Exited (0) About an hour ago
   elastic_jennings

C:\Users\CEDDU>
```

docker logs – For Debugging

```
C:\Users\CEDDU>docker logs e8b19a306c0d
Starting Redis Server
1:C 21 Feb 2026 06:02:18.302 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
1:C 21 Feb 2026 06:02:18.302 * Redis version=8.6.0, bits=64, commit=00000000,
1:C 21 Feb 2026 06:02:18.302 * Configuration loaded
1:M 21 Feb 2026 06:02:18.302 * monotonic clock: POSIX clock_gettime
1:M 21 Feb 2026 06:02:18.303 * Running mode=standalone, port=6379.
1:M 21 Feb 2026 06:02:18.304 * <bf> RedisBloom version 8.6.0 (Git=unknown)
1:M 21 Feb 2026 06:02:18.304 * <bf> Registering configuration options: [
```

Container Names

```
C:\Users\CEDDU>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS      NAMES
bbf3a6231466   redis     "docker-entrypoint.s…"   17 minutes ago   Up 6 minutes   6379/tcp   serene_hamilton

C:\Users\CEDDU>docker stop bbf3a6231466
bbf3a6231466

C:\Users\CEDDU>docker run -d --name redis-container redis
d6b3bc435e3d087448213a57649917fcc7c93b01536d699970b791b57becc1a8

C:\Users\CEDDU>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS      NAMES
d6b3bc435e3d   redis     "docker-entrypoint.s…"   9 seconds ago   Up 6 seconds   6379/tcp   redis-container

C:\Users\CEDDU>
```

Docker: exec command

```
C:\Users\CEDDU>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS      NAMES
d6b3bc435e3d   redis     "docker-entrypoint.s…"   4 minutes ago    Up 4 minutes    6379/tcp   redis-container
```

```
C:\Users\CEDDU>docker exec -it d6b3bc435e3d /bin/bash
root@d6b3bc435e3d:/data# ls
root@d6b3bc435e3d:/data# pwd
/data
root@d6b3bc435e3d:/data# cd /
root@d6b3bc435e3d:/# pwd
/
root@d6b3bc435e3d:/#
```

docker run vs. docker start

- Docker run command takes image as input and creates a new container
- Docker start command takes container as input and starts the container

Dockerfile

- A Dockerfile is a text-based document that's used to create a container image.
- It provides instructions to the image builder on the commands to run, files to copy, startup command, and more.

Writing a Dockerfile

Dockerfile > ...

```
1   #getting base image
2   FROM ubuntu
3
4   #RUN is executed during building of an Image
5   RUN apt-get update
6
7   #CMD is executed when a container is created from the Image
8   CMD [ "echo", "Hello World! From Docker Image" ]
```

Dockerfile

Instruction	Description
FROM <image>	Defines a base for your image
RUN <command>	Executes any commands in a new layer on top of the current image and commits the result. RUN also has a shell form for running commands
WORKDIR <directory>	Sets the working directory for any RUN, CMD, ENTRYPOINT, COPY, and ADD instructions that follow it in the Dockerfile
COPY <src> <dest>	Copies new files or directories from <src> and adds them to the filesystem of the container at the path <dest>
CMD <command>	Lets you define the default program that is run once you start the container based on this image. Each Dockerfile only has one CMD, and only the last CMD instance is respected when multiple exist
EXPOSE <port>	container binds and listens on a specified port at runtime

Building an Image

```
C:\Users\DELL\Docker>docker build .
[+] Building 14.6s (5/6)
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 255B
 => [internal] load metadata for docker.io/library/ubuntu:late
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/2] FROM docker.io/library/ubuntu:latest@sha256:72297848
 => => resolve docker.io/library/ubuntu:latest@sha256:72297848
 => [auth] library/ubuntu:pull token for registry-1.docker.io
 => [2/2] RUN apt-get update
```

Building an Image

```
[+] Building 71.3s (7/7) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 255B
 => [internal] load metadata for docker.io/library/ubuntu:latest
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/2] FROM docker.io/library/ubuntu:latest@sha256:72297848456
 => => resolve docker.io/library/ubuntu:latest@sha256:72297848456
 => [auth] library/ubuntu:pull token for registry-1.docker.io
 => [2/2] RUN apt-get update
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:f39efd4379b0697996e174fd32fb307d
 => => exporting config sha256:0e9a7d287d218af4540617fea3647034a3
 => => exporting attestation manifest sha256:930d9386ccbe2bea9a78
 => => exporting manifest list sha256:d2805960f9875e36754d338697b
 => => naming to moby-dangling@sha256:d2805960f9875e36754d338697b
 => => unpacking to moby-dangling@sha256:d2805960f9875e36754d3386
```

View Created Image

```
C:\Users\DELL\Docker>docker images
REPOSITORY   TAG       IMAGE ID       CREATED          SIZE
<none>       <none>    d2805960f987   5 minutes ago    195MB
postgres     17.2      3267c505060a   5 days ago       614MB
postgres     latest    3267c505060a   5 days ago       614MB
ubuntu       latest    72297848456d   13 days ago      117MB
mysql        latest    d56d039139a7   2 weeks ago      1.09GB
redis        latest    93a8d83b707d   4 weeks ago      173MB
```

Create Image with Tag Name

```
C:\Users\DELL\Docker>docker build -t myimage1:1.0 .
[+] Building 1.8s (6/6) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 255B
 => [internal] load metadata for docker.io/library/ubuntu:latest
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/2] FROM docker.io/library/ubuntu:latest@sha256:72297848456
 => => resolve docker.io/library/ubuntu:latest@sha256:72297848456
 => CACHED [2/2] RUN apt-get update
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:f39efd4379b0697996e174fd32fb307c
 => => exporting config sha256:0e9a7d287d218af4540617fea3647034a3
 => => exporting attestation manifest sha256:70763760e652051879af
 => => exporting manifest list sha256:6078dcd9c7359d0b99fe6b1b67c
 => => naming to docker.io/library/myimage1:1.0
 => => unpacking to docker.io/library/myimage1:1.0
```

View Created Image

```
C:\Users\DELL\Docker>docker images
REPOSITORY   TAG       IMAGE ID       CREATED          SIZE
myimage1     1.0       6078dcd9c735   7 minutes ago    195MB
<none>       <none>    d2805960f987   7 minutes ago    195MB
postgres     17.2      3267c505060a   5 days ago       614MB
postgres     latest    3267c505060a   5 days ago       614MB
ubuntu       latest    72297848456d   13 days ago      117MB
mysql        latest    d56d039139a7   2 weeks ago      1.09GB
redis        latest    93a8d83b707d   4 weeks ago      173MB
```

Run the Created Image

```
C:\Users\DELL\Docker>docker images
REPOSITORY   TAG        IMAGE ID       CREATED             SIZE
<none>       <none>     d2805960f987   12 minutes ago      195MB
myimage1     1.0        6078dcd9c735   12 minutes ago      195MB
postgres     17.2       3267c505060a   5 days ago          614MB
postgres     latest     3267c505060a   5 days ago          614MB
ubuntu       latest     72297848456d   13 days ago         117MB
mysql        latest     d56d039139a7   2 weeks ago         1.09GB
redis        latest     93a8d83b707d   4 weeks ago         173MB

C:\Users\DELL\Docker>docker run 6078dcd9c735
Hello World! From Docker Image
```

Run the Created Image

```
C:\Users\DELL\Docker>docker run -it myimage1:1.0 /bin/bash
root@c7de52cb1a4d:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc
root@c7de52cb1a4d:/# pwd
/
root@c7de52cb1a4d:/# cat
hi
hi
^C
root@c7de52cb1a4d:/# exit
exit

C:\Users\DELL\Docker>
```

Containers

Give feedback

View all your running containers and applications. Learn more

Container CPU usage

No containers are running.

Container memory usage

No containers are running.

Search

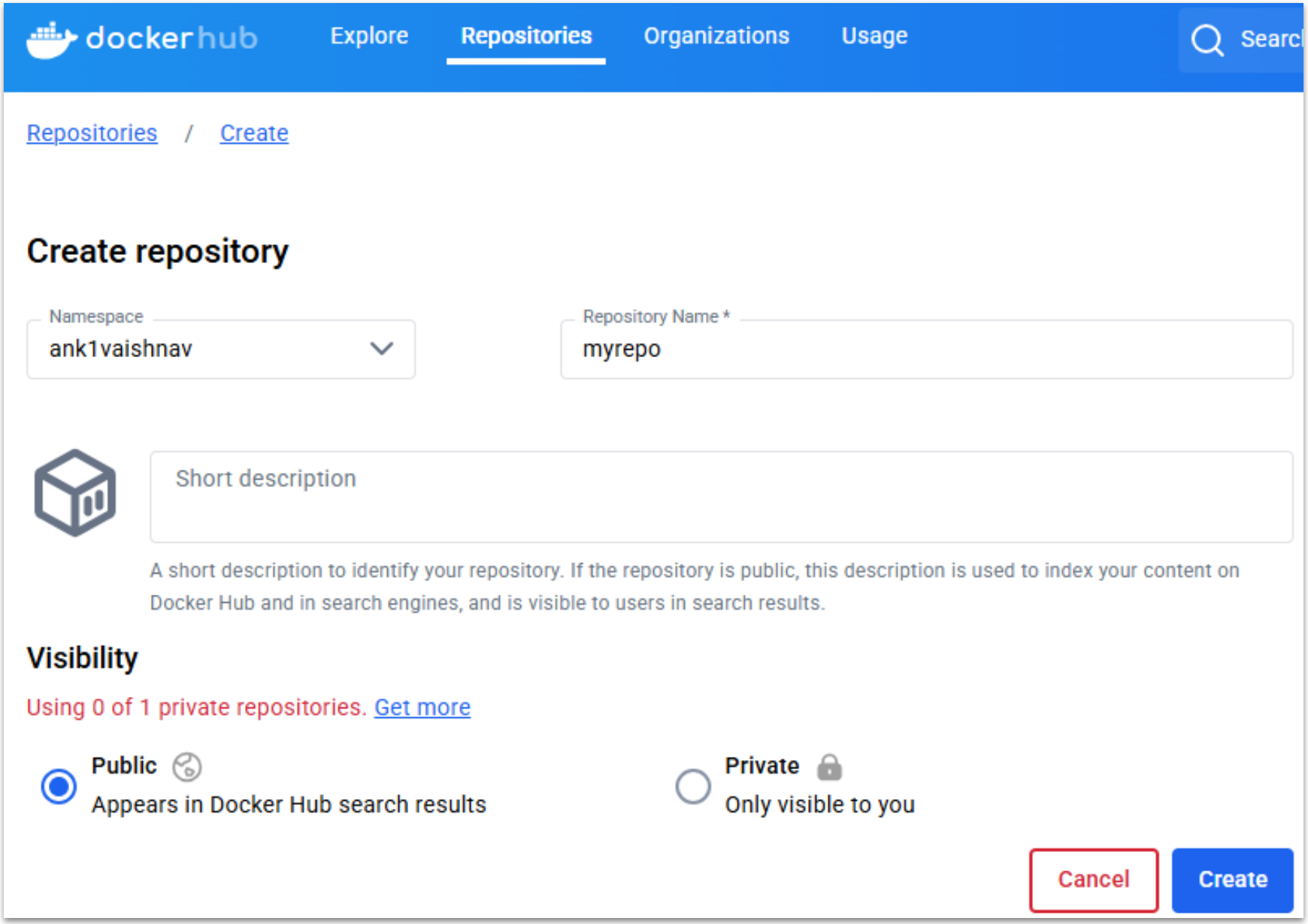
Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last started
☐	pensive_tharp	97fd4318442b	6078dcd9c735		N/A	4 minutes ago

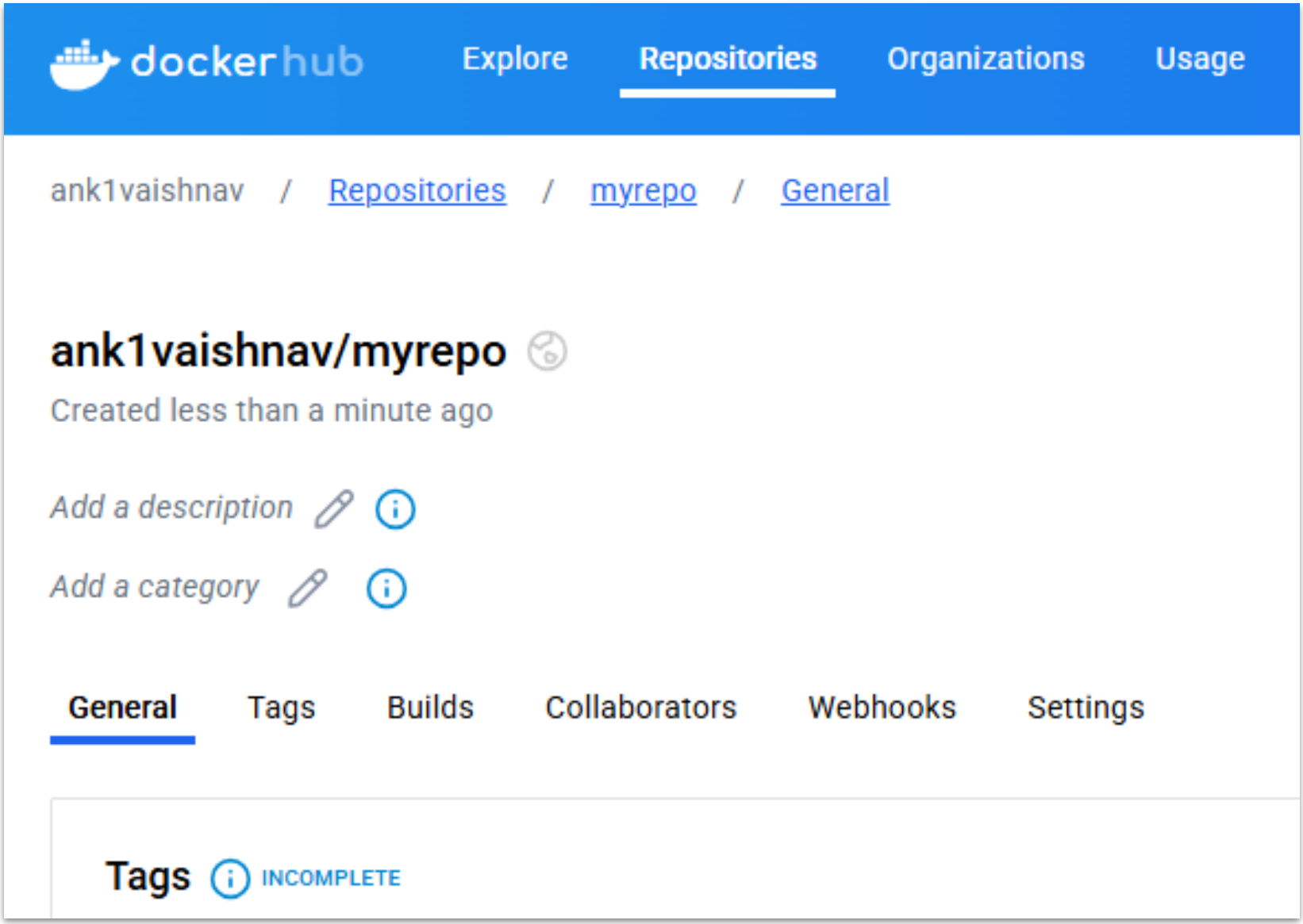
Steps to work with Dockerfile

- Create a file named Dockerfile
- Add instructions in the Dockerfile
- Build Dockerfile to create an Image
- Run the Image to create a Container

Docker Hub: Create Repository



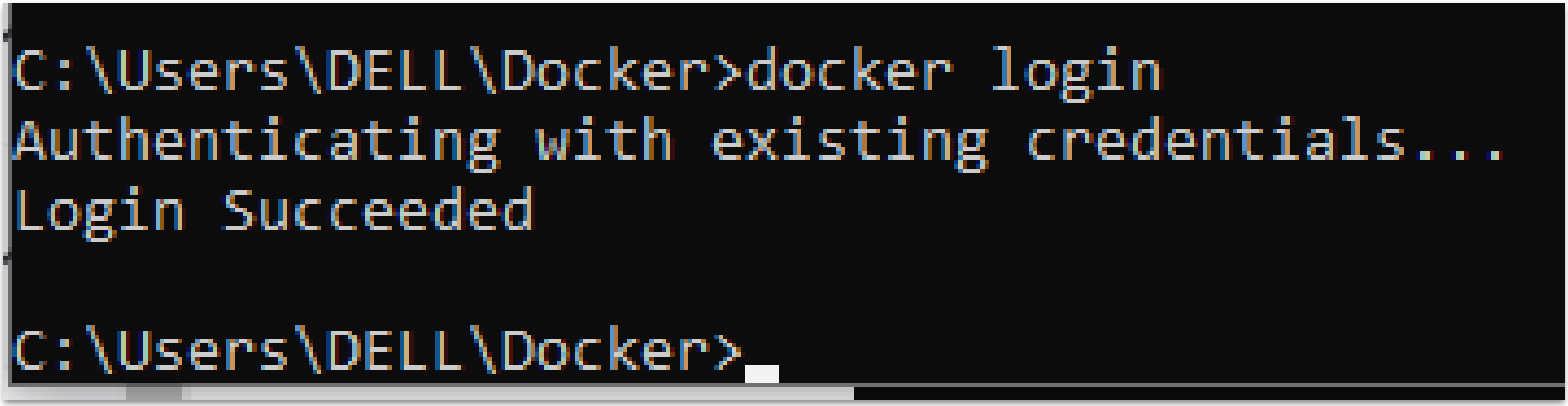
Docker Hub: Create Repository



Docker Hub: Create Image

```
C:\Users\DELL\Docker>docker build -t ank1vaishnav/myrepo .
[+] Building 5.0s (7/7) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 255B
 => [internal] load metadata for docker.io/library/ubuntu:latest
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/2] FROM docker.io/library/ubuntu:latest@sha256:72297848456
 => => resolve docker.io/library/ubuntu:latest@sha256:72297848456
 => [auth] library/ubuntu:pull token for registry-1.docker.io
 => CACHED [2/2] RUN apt-get update
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:f39efd4379b0697996e174fd32fb307d
 => => exporting config sha256:0e9a7d287d218af4540617fea3647034a3
 => => exporting attestation manifest sha256:30708760cf82fc4a6e52
 => => exporting manifest list sha256:85e168829983 2318c4d3cd91bb6
 => => naming to docker.io/ank1vaishnav/myrepo:latest
 => => unpacking to docker.io/ank1vaishnav/myrepo:latest
```

Docker Hub: Login

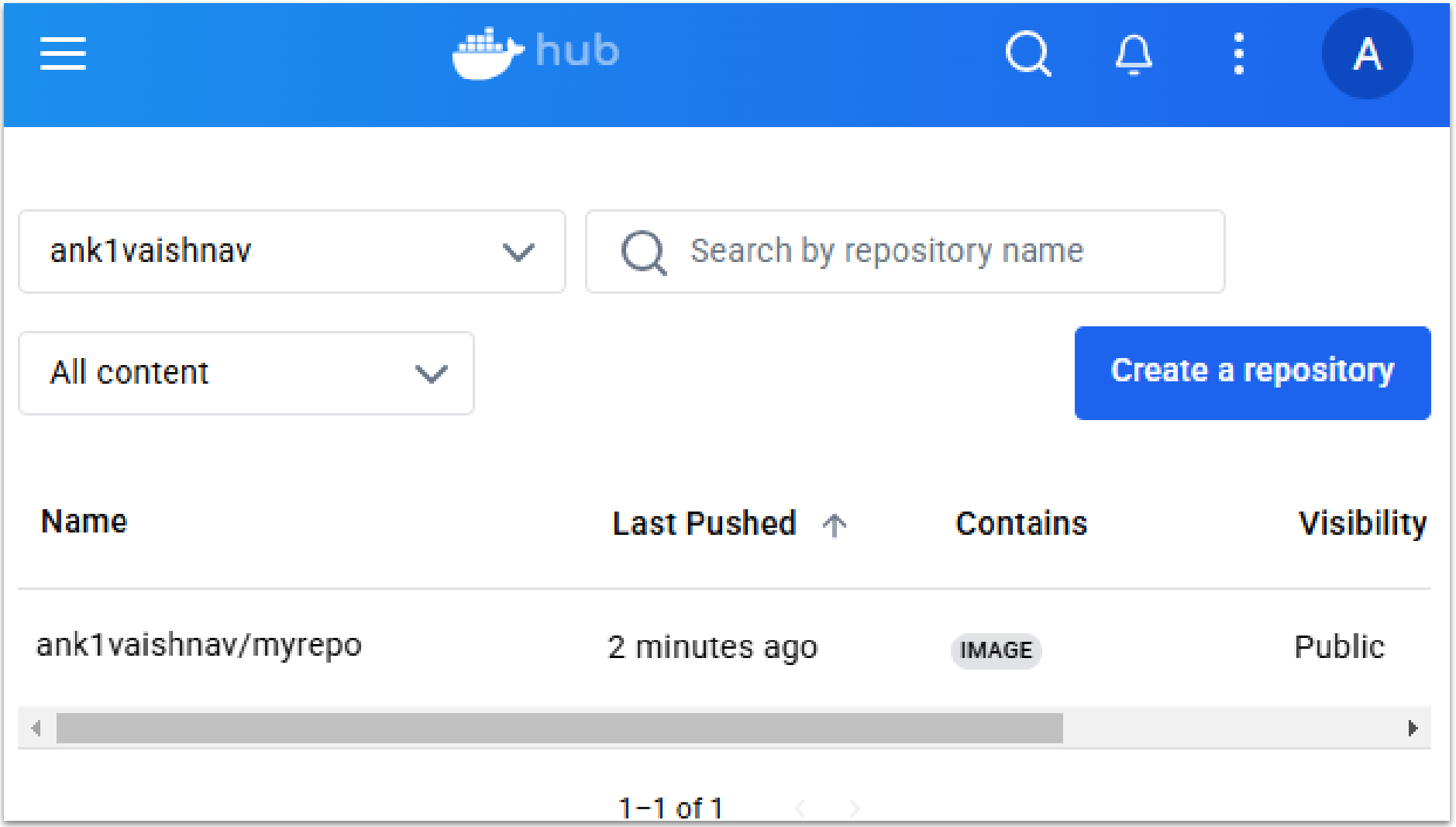


Docker Hub: Push Image

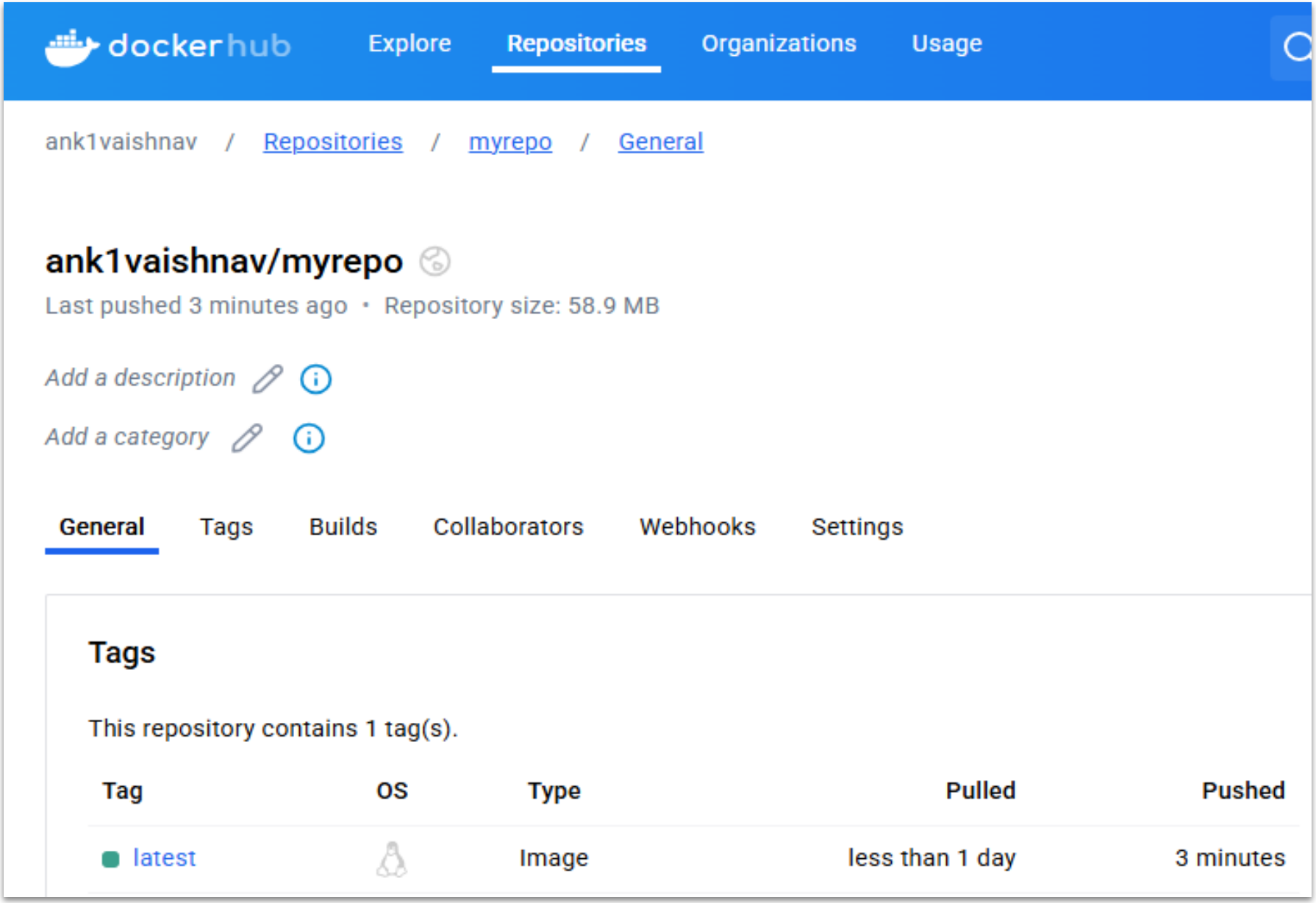
```
C:\Users\DELL\Docker>docker push ank1vaishnav/myrepo
Using default tag: latest
The push refers to repository [docker.io/ank1vaishnav/myrepo]
5a7813e071bf: Mounted from library/ubuntu
2f0a1068056e: Pushed
082bbfcf349e: Pushed
latest: digest: sha256:85e1688299832318c4d3cd91bb6076c7d5dfdad

C:\Users\DELL\Docker>
```

Docker Hub: Push Image



Docker Hub: Push Image



Docker Hub: Pull Image

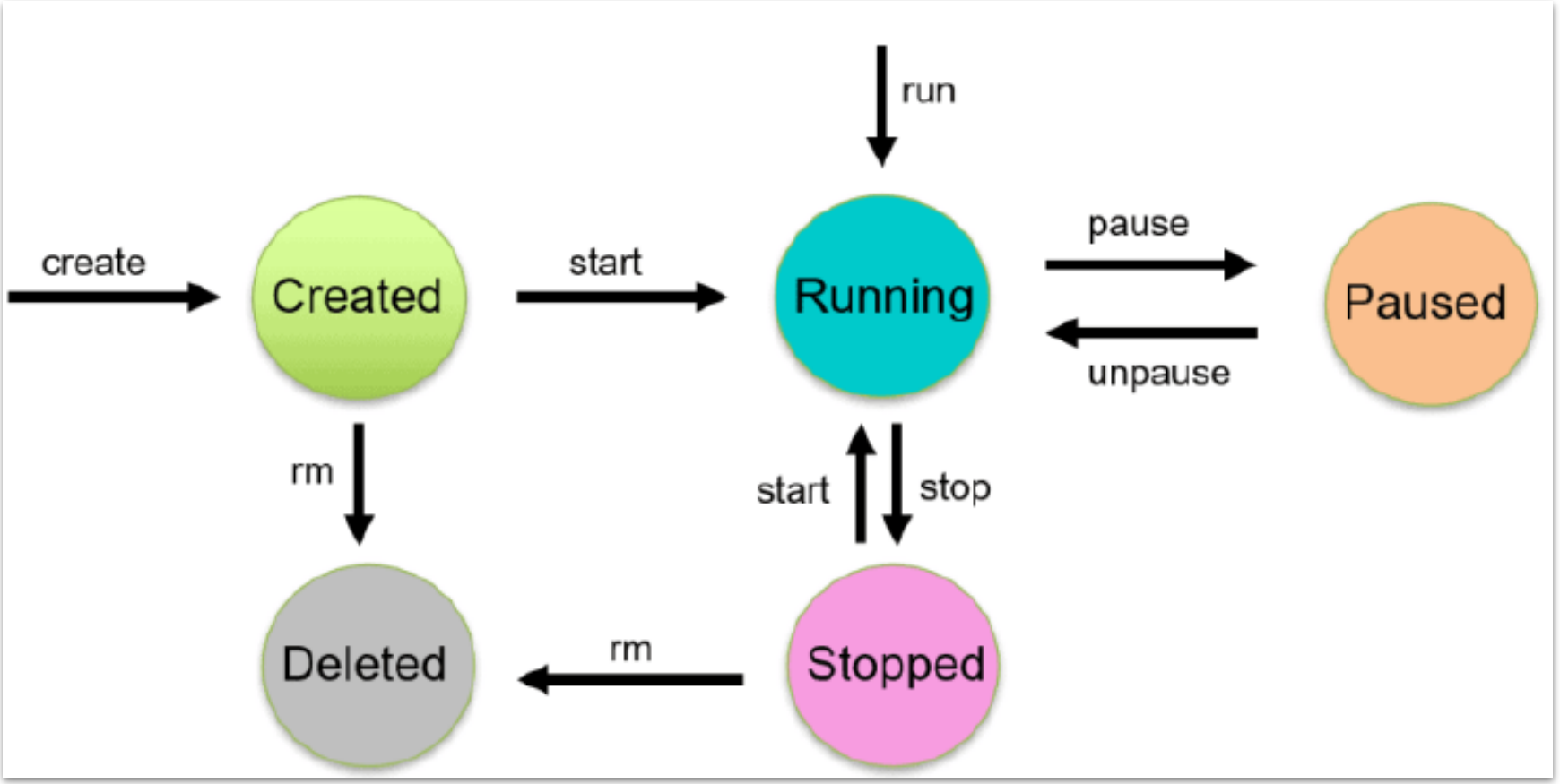
```
C:\Users\DELL\Docker>docker pull ank1vaishnav/myrepo:latest
latest: Pulling from ank1vaishnav/myrepo
Digest: sha256:85e1688299832318c4d3cd91bb6076c7d5dfdad3cb738
Status: Image is up to date for ank1vaishnav/myrepo:latest
docker.io/ank1vaishnav/myrepo:latest
```

Docker Hub: Run

```
C:\Users\DELL\Docker>docker images
REPOSITORY                TAG       IMAGE ID       CREATED         SIZE
<none>                    <none>    d2805960f987   2 days ago      195MB
myimage1                  1.0       6078dcd9c735   2 days ago      195MB
ank1vaishnav/myrepo       latest    85e168829983   2 days ago      195MB
postgres                  17.2      3267c505060a   8 days ago      614MB
postgres                  latest    3267c505060a   8 days ago      614MB
ubuntu                    latest    72297848456d   2 weeks ago     117MB
mysql                     latest    d56d039139a7   2 weeks ago     1.09GB
redis                     latest    93a8d83b707d   5 weeks ago     173MB

C:\Users\DELL\Docker>docker run ank1vaishnav/myrepo
Hello World! From Docker Image
```

Container Lifecycle



Pause Container

```
C:\Users\DELL\Docker>docker ps
CONTAINER ID   IMAGE          COMMAND       CREATED      STATUS         PORTS     NAMES
c7de52cb1a4d   myimage1:1.0   "/bin/bash"   2 days ago   Up 8 seconds             mystifying_burnell
```

```
C:\Users\DELL\Docker>docker pause c7de52cb1a4d
c7de52cb1a4d

C:\Users\DELL\Docker>docker ps
CONTAINER ID   IMAGE          COMMAND       CREATED      STATUS                   PORTS     NAMES
c7de52cb1a4d   myimage1:1.0   "/bin/bash"   2 days ago   Up 3 minutes (Paused)              mystifying_burnell
```

Stop and Remove a Container

```
C:\Users\DELL\Docker>docker stop c7de52cb1a4d
c7de52cb1a4d

C:\Users\DELL\Docker>docker ps -a
CONTAINER ID   IMAGE                 COMMAND                  CREATED       STATUS
   NAMES
67715dc1af69   ank1vaishnav/myrepo   "echo 'Hello World! …"   5 hours ago   Exited (0) 5 hours ago
   inspiring_keller
c7de52cb1a4d   myimage1:1.0          "/bin/bash"              2 days ago    Exited (137) 12 seconds ago
   mystifying_burnell
97fd4318442b   6078dcd9c735          "echo 'Hello World! …"   2 days ago    Exited (0) 2 days ago
   pensive_tharp
```

```
C:\Users\DELL\Docker>docker rm 97fd4318442b
97fd4318442b

C:\Users\DELL\Docker>docker ps -a
CONTAINER ID   IMAGE                 COMMAND                  CREATED       STATUS
  NAMES
67715dc1af69   ank1vaishnav/myrepo   "echo 'Hello World! …"   5 hours ago   Exited (0) 5 hours ago
  inspiring_keller
c7de52cb1a4d   myimage1:1.0          "/bin/bash"              2 days ago    Exited (137) 6 minutes ago
  mystifying_burnell
```

Docker Volumes

- Docker containers are easiest to use with stateless applications because their filesystems are ephemeral in nature.

- Changes made to a container's environment are lost when the container stops, crashes, or gets replaced.

Docker Volumes

- Stateful applications such as databases and file servers can be dockerized by attaching volumes to the containers.
- Volumes provide persistent storage that's independent of individual containers.
- Volumes can be reattached to a different container after a failure or can be used to share data between several containers simultaneously.

Docker Volumes

- Volumes are a mechanism for storing data outside containers.

- All volumes are managed by Docker and stored in a dedicated directory on the host, usually /var/lib/docker/volumes for Linux systems.

Docker Volumes

- Volumes are mounted to filesystem paths in the containers.
- When containers write to a path beneath a volume mount point, the changes will be applied to the volume instead of the container's writable image layer.
- The written data will still be available if the container stops – as the volume's stored separately on the host, it can be remounted to another container or accessed directly using manual tools.

Bind Mounts vs. Docker Volumes

- Bind mounts are another way to give containers access to files and folders on the host.
- They directly mount a host directory into the container.
- Bind mounts are best used for ad-hoc storage on a short-term basis.

Bind Mounts vs. Docker Volumes

- Volumes are a better solution when it is needed to provide permanent storage to operational containers.
- Because they're managed by Docker, it is not needed to manually maintain directories on the host.
- There's less chance of data being accidentally modified and no dependency on a particular folder structure.

Docker Volume Use cases

- **Database storage** – Volume can be mounted to the storage directories used by databases such as MySQL, Postgres, and Mongo. This will ensure that data persists after the container stops.
- **Application data** – Data generated by the application, such as file uploads, documents, and profile photos, can be stored in a volume.
- **Essential caches** – Volume can be used to persist the contents of any caches, which would take significant time to rebuild.

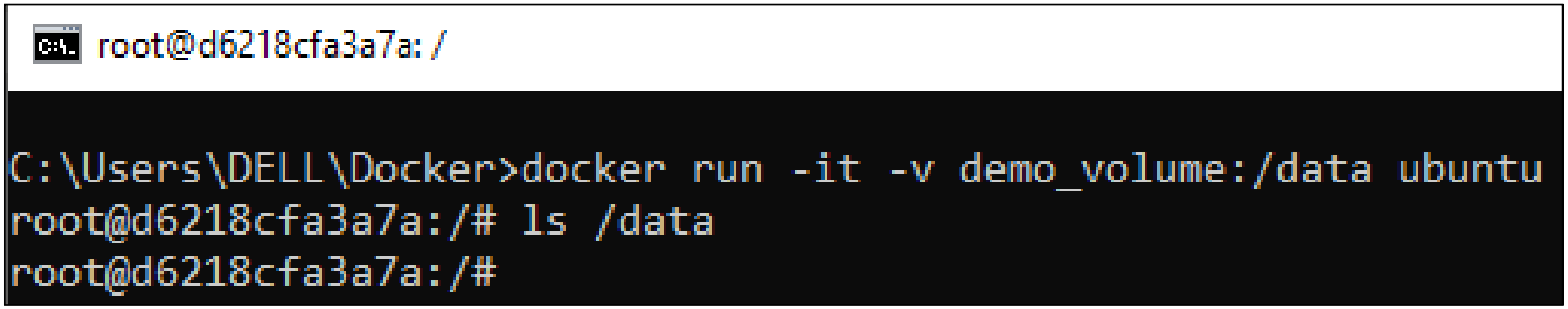
Docker Volume Use cases

- **Convenient data backups** – Docker's centralized volume storage makes it easy to backup container data by mirroring /var/lib/docker/volumes to another location.
- **Share data between containers** – Docker volumes can be mounted to multiple containers simultaneously.
- **Write to remote filesystems** – You'll need to use a volume when you want containers to write to remote filesystems

Using Docker Volumes

- A container can be started with a volume by setting the -v flag when docker run is called.

- The following command starts a new Ubuntu 22.04 container and attaches the terminal to it (-it), A volume called demo_volume is mounted to /data inside the container.



Using Docker Volumes

```
root@d6218cfa3a7a: /

C:\Users\DELL\Docker>docker run -it -v demo_volume:/data ubuntu
root@d6218cfa3a7a:/# ls /data
root@d6218cfa3a7a:/#
```

```
C:\Users\DELL>docker ps
CONTAINER ID   IMAGE     COMMAND       CREATED         STATUS         PORTS     NAMES
d6218cfa3a7a   ubuntu    "/bin/bash"   6 minutes ago   Up 6 minutes             infallible_ritchie
```

```
root@d6218cfa3a7a:/# echo "foobar" > /data/foo
root@d6218cfa3a7a:/# cat /data/foo
foobar
root@d6218cfa3a7a:/# exit
exit

C:\Users\DELL\Docker>_
```

Using Docker Volumes

- Start a new container that attaches the same volume.

```
C:\Users\DELL\Docker>docker run -it -v demo_volume:/app ubuntu
```

- The demo_volume already exists so docker will reuse it instead of creating a new one.

```
root@2ac84fb23a35:/# cat /app/foo
foobar
root@2ac84fb23a35:/# exit
exit
```

- Docker persisted the volume's content after the first container stopped, allowing it to be reused with your replacement container.

Create Docker Volume Manually

- Docker volume can be manually created with the docker volume create command.

```
C:\Users\DELL\Docker>docker volume create app_data
app_data
```

- The volume can then be mounted to containers in the same way as before

```
C:\Users\DELL\Docker>docker run -it -v app_data:/app ubuntu
root@0ec118384259:/# ls
app  bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc
root@0ec118384259:/# pwd
/
```

Using Volumes in Dockerfile

- Docker allows images to define volume mount points with the VOLUME Dockerfile instruction.
- When a container is started from an image, Docker will automatically create new volumes for the mount points listed in the Dockerfile.

```
FROM ubuntu:22.04
VOLUME /app_data
```

Interacting with Docker Volumes

- The Docker CLI includes a set of commands for interacting with the volumes on the host.

- List all volumes with docker volume ls:

```
C:\Users\DELL\Docker>docker volume ls
DRIVER    VOLUME NAME
local     710b01c8e03a90d8c4cd4d713a2a
local     app_data
local     demo_volume
```

Interacting with Docker Volumes

- To access more detailed information about a specific volume, use docker volume inspect.

```
C:\Users\DELL\Docker>docker volume inspect demo_volume
[
    {
        "CreatedAt": "2025-02-13T13:48:35Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/demo_volume/_data",
        "Name": "demo_volume",
        "Options": null,
        "Scope": "local"
    }
]
```

Interacting with Docker Volumes

- Delete a volume with docker volume rm:

```
C:\Users\DELL\Docker>docker volume remove app_data
app_data

C:\Users\DELL\Docker>docker volume ls
DRIVER    VOLUME NAME
local     710b01c8e03a90d8c4cd4d713a2a38bdd09a04f34
local     demo_volume

C:\Users\DELL\Docker>
```

Docker Compose

- Docker Compose is a tool that makes it easier to create and run multi-container applications.

- It automates the process of managing several Docker containers simultaneously, such as a website frontend, API, and database service.

Docker Compose

- Docker Compose allows to define the application's containers as code inside a YAML file.

- Once the file (normally named docker-compose.yml) is created, all the containers (called "services") can be started with a single Compose command.

Docker Compose

- Compared with manually starting and linking containers, Compose is quicker, easier, and more repeatable.

- The containers will run with the same configuration every time—there's no risk of forgetting to include an important docker run flag.

Why Docker Compose?

- Most real-world applications have several services with dependency relationships—
  - for example, the app may run in one container, but depend on a database server that's deployed adjacently in another container.

- Moreover, services usually need to be configured with storage volumes, environment variables, port bindings, and other settings before they can be deployed.

Docker Vs. Docker Compose

- Docker is a containerization engine that provides a CLI for building, running, and managing individual containers on the host.

- Compose is a tool that expands Docker with support for multi-container management.
  - It supports “stacks” of containers that are declaratively defined in project-level config files.

Docker Vs. Docker Compose

- Docker can be used without Compose; however, adopting Compose when developing a containerized system allows to deploy the app in any environment with a single command.

- Whereas the Docker CLI only interacts with one container at a time, Compose integrates with the project and is aware of the relationships between your containers.

Docker Compose Benefits

- Fast and easy configuration with YAML scripts
- Single host deployment
- Increased productivity
- Security with isolated containers